iReside

CLOUD INFRASTRUCTURE & SYSTEMS ARCHITECTURE

IT TECHNICAL DOCUMENTATION & SYSTEM ADMINISTRATION HANDBOOK

Production Topology, Security Invariants, Database Schema & Operational Runbooks



DOCUMENT ID	DOC REF: IR-IT-2026-V1.0
CLASSIFICATION	Internal Technical Standard • Production Reference
AUDIENCE	Systems Architects, DevOps Engineers, and Database Administrators
STACK	Next.js 16 (React 19), Supabase PostgreSQL 15, Vercel & Capacitor 7
PILOT SCOPE	Barangay Marulas Pilot Implementation, Valenzuela City

Table of Contents

1. System Architecture & Complete Tech Stack **Page 3**
2. Strict Two-Role Model & Zero Admin Architecture **Page 4**
3. Environment Variables & Secrets Governance **Page 5**
4. Database Schema: Users, Profiles & Buildings **Page 6**
5. Database Schema: Leases, Financials & Operations **Page 7**
6. Row-Level Security (RLS) Policy Governance **Page 8**
7. Authentication Engine & 2FA Session Guards **Page 9**
8. Cryptographic Emergency Spare Key Engine **Page 10**
9. Turnkey Workspace Launch Lifecycle (/api/setup) **Page 11**
10. Core Application REST API Directory **Page 12**
11. Mobile Client Architecture & Android Build Pipeline **Page 13**
12. Automated Schedulers & Serverless Cron Workers **Page 14**
13. Database Backups, Disaster Recovery & Runbooks **Page 15**
14. Commissioning Sign-Off, Support & Glossary **Page 16**

Executive Architecture Overview

This technical manual establishes production architecture and security invariants.

The platform decouples presentation, serverless edge compute, and managed relational persistence.

The system enforces a strict two-role model: **Landlord** and **Tenant** with zero human admin.

All automated tasks execute through secure service role tokens.

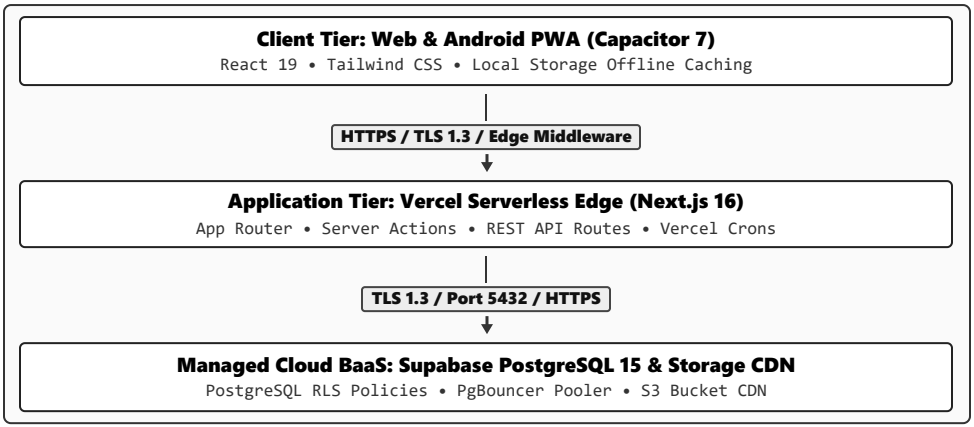
System Engineering Invariants & SLA Metrics

METRIC / INVARIANT	PRODUCTION STANDARD	ARCHITECTURAL JUSTIFICATION
Role Architecture	Strict Two-Role: Landlord & Tenant	External entities communicate through processes.
Service Role Key	Serverless Backend Task Automation	Bypasses client RLS for background jobs.
Uptime Objective	99.9% Production High Availability	Redundant edge infrastructure with database backups.
Legal Standards	RA 10173 and RA 8792 Compliance	Encryption at rest and binding signatures.

> **Engineering Standard:** All architectural changes must preserve passing automated test suites.

Production Topology Architecture

iReside deploys a modern three-tier serverless topology with edge acceleration:



Complete Technology Stack Specifications

SUBSYSTEM	TECHNOLOGY	RUNTIME / ROLE
Frontend	Next.js 16 (App Router)	React 19 Server Components and Server Actions
Language	TypeScript 5.x	Strict mode, end-to-end type safety
Styling	Tailwind CSS v3.4	Mobile-responsive, high-contrast accessible
Database	PostgreSQL 15.x	Supabase BaaS with Row-Level Security
Connection Pool	PgBouncer	Transaction connection pooling on port 6543
Mobile	Capacitor 7 Native Wrapper	Android SDK 34, standalone Gradle project
File Storage	Supabase Storage	S3-compatible encrypted CDN buckets
Edge Compute	Vercel Serverless	Edge middleware and serverless functions

Network Ingress Routing & Protocol Bindings

TRAFFIC LAYER	PROTOCOL / PORT	DESTINATION SUBSYSTEM
Web & Mobile Traffic	HTTPS / TLS 1.3 (Port 443)	Vercel Edge Serverless Gateway.
Database Pool	TCP Port 6543 (PgBouncer)	PostgreSQL 15 Managed Cluster.
Object Storage CDN	HTTPS API (Port 443)	Encrypted S3-compatible buckets.

Strict Two-Role Human User Model

Per system rules, iReside implements a strict two-role model with **zero admin role**:

- **Landlord:** Property owner. Manages properties, unit maps, leases, invoices, and maintenance.
- **Tenant:** Resident occupant. Signs digital leases, submits payments, logs maintenance tickets.

Service Role Operational Model

Automated processes that require cross-tenant or system-level access use secure service role tokens:

- **Automated Billing Cron:** Daily midnight worker generates recurring rent and overdue charges.
- **Workspace Initialization (/api/setup/launch):** Bypasses client RLS to stamp initial configuration.
- **Disaster Recovery & Seeding:** Dedicated CLI maintenance scripts run via service role key.

Access Control Matrix

RESOURCE / SCOPE	LANDLORD	TENANT	SERVICE ROLE
Property Configuration	Read / Write (Own)	Read Only (Assigned)	Full Bypass
Lease Contracts	Create / Sign (Own)	Sign / View (Own)	Full Bypass
Payment Verification	Verify / Reject	Submit / View (Own)	Automated Audit
System Initialization	Execute Launch	No Access	Atomic Write
Submeter Readings	Record & Bill (Own)	View Bills (Assigned)	Automated Billing
Repair Work Orders	Assign & Close (Own)	Create & View (Own)	Work Order Worker
Emergency Spare Key	Reset & Generate (Own)	Reset & Generate (Own)	Bcrypt Hash Engine

Human Role vs. Service Role Boundary Specifications

SECURITY BOUNDARY	DESIGN CONSTRAINT	ENFORCEMENT ARCHITECTURE
Zero Human Admin	No master user exists.	Database lacks admin enum values.
Service Automation	Background tasks use tokens.	Bypasses RLS through server credentials.
Role Segregation	Landlords cannot access tenants.	Strict foreign key tenant checks.

> **Data Flow Invariant:** External entities never communicate directly without mediating processes.

Required Environment Variables (.env.local)

All production instances must supply the following environment secrets in `.env.local`:

VARIABLE	SCOPE	DESCRIPTION
`NEXT_PUBLIC_SUPABASE_URL`	Public / Browser	Supabase HTTPS project endpoint
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Public / Browser	Client-side anonymous key (RLS enforced)
`SUPABASE_SERVICE_ROLE_KEY`	Server Only	Bypasses RLS for setup, launch, and crons
`CRON_SECRET`	Server Only	Bearer token authenticating Vercel cron triggers
`NEXT_PUBLIC_APP_URL`	Public / Browser	Production canonical URL for invite generation
`DATABASE_URL`	Server Only	Direct connection for migrations and seeds
`CAPACITOR_SERVER_URL`	Public / Build	Canonical web URL for Android APK
`NEXT_PUBLIC_APP_ENV`	Public / Browser	Deployment target environment name
`RESEND_API_KEY`	Server Only	Transactional email delivery for 2FA OTP codes.
`JWT_EXPIRY_SECONDS`	Server Only	Session token lifecycle duration setting.

Environment Secret Rotation & Key Lifecycle Standards

SECRET KEY	ROTATION PERIOD	ROTATION RUNBOOK
SERVICE_ROLE_KEY	Annual or post-incident	Rotate in Supabase dashboard and Vercel.
CRON_SECRET	Semi-annually	Generate 32-byte hex string in CLI.
ANON_KEY	Major release cycles	Bound to client browser environment.

Secret Isolation & Rotation Guidelines

- **Service Role Key Isolation:** Never prefix `SUPABASE_SERVICE_ROLE_KEY` with `NEXT_PUBLIC_`.
- **Git Hygiene:** Ensure `.env*` patterns remain in `.gitignore`.
- **Zero Compromise:** Compromise of the service role key grants full unrestricted database write access.

> **Secret Governance: Never commit environment files to public version control.**

Entity Hierarchy: Properties & 2D Floor Plans

Properties represent rental buildings in Barangay Marulas with floor levels and individual units:

TABLE	PRIMARY KEY	FOREIGN KEYS	KEY COLUMNS
`profiles`	`id (UUID)`	`auth.users(id)`	`role`, `full_name`, `phone`, `emergency_spare_key`
`properties`	`id (UUID)`	`landlord_id → profiles(id)`	`name`, `address`, `total_units`, `gcash_qr_url`
`floors`	`id (UUID)`	`property_id → properties(id)`	`floor_number`, `label`, `display_order`
`units`	`id (UUID)`	`floor_id → floors(id)`	`unit_number`, `monthly_rent`, `status` (vacant, occupied)

Relational Integrity & Foreign Key Cascade Actions

PARENT TABLE	CHILD TABLE	FOREIGN KEY COLUMN	CASCADE POLICY
`profiles`	`properties`	`landlord_id`	`ON DELETE RESTRICT`
`properties`	`floors`	`property_id`	`ON DELETE CASCADE`
`floors`	`units`	`floor_id`	`ON DELETE CASCADE`
`units`	`leases`	`unit_id`	`ON DELETE RESTRICT`

Unit Map Coordinate Layout Constraints

LAYOUT ATTRIBUTE	DATA TYPE	DATABASE CONSTRAINT
`grid_x, grid_y`	`INTEGER`	Clamped to non-negative grid coordinates.
`layout_preset`	`VARCHAR(30)`	Checked against valid architectural presets.
`status`	`VARCHAR(20)`	B-tree index for rapid vacancy filtering.

Unit Status Enum Transitions & Availability Rules

STATUS ENUM	BUSINESS CONDITION	AUTOMATED TRANSITION RULE
`vacant`	Unit has no active lease.	Available for tenant application assignment.
`occupied`	Active signed tenancy lease.	Locked against duplicate resident assignment.
`maintenance`	Active critical repair order.	Temporarily hidden from tenant listings.

Financial Engine & Ledger Relational Architecture

iReside models the complete tenancy lifecycle from digital signing to submetered utility invoicing:

TABLE	FOREIGN KEYS	KEY COLUMNS	ENUMS / CONSTRAINTS
`leases`	`unit_id`, `tenant_id`	`start_date`, `end_date`, `rent_amount`	`status`: active, pending, ended
`invoices`	`lease_id`, `property_id`	`amount_due`, `due_date`, `billing_period`	`status`: unpaid, pending, paid
`payments`	`invoice_id`, `tenant_id`	`amount_paid`, `reference_no`, `proof_url`	`method`: gcash, cash, maya
`maintenance_tickets`	`unit_id`, `tenant_id`	`category`, `description`, `photo_url`	`status`: pending, in_progress, completed
`utility_readings`	`unit_id`, `property_id`	`meter_type`, `current_reading`, `total_cost`	`meter_type`: electricity, water
`announcements`	`property_id`, `author_id`	`title`, `content`, `category`	Broadcast to property residents

Financial State Machine & Invoice Lifecycle

STATE TRANSITION	TRIGGER EVENT	LEDGER BALANCE IMPACT
`Null → unpaid`	Daily billing cron runs	Posts monthly invoice debit.
`unpaid → pending`	Tenant uploads proof	Places receipt in review queue.
`pending → paid`	Landlord confirms receipt	Credits balance and settles invoice.
`unpaid → overdue`	Due date passes grace period	Flags account for collection action.

Payment Rail Ledger Validation Protocols

PAYMENT RAIL	VALIDATION CHECK	ACCOUNTING LEDGER ACTION
GCash / Maya	13-digit reference checksum.	Automated receipt creation upon approval.
Direct Cash	Physical cash receipt verification.	Instant ledger credit stamp.
Bank Transfer	Account transfer confirmation slip.	Reconciled against monthly bank statements.

> **Data Durability:** Financial transactions utilize ACID compliance to prevent double settlements.

PostgreSQL Row-Level Security (RLS) Architecture

Every table in the iReside database enforces strict Row-Level Security. Unauthenticated queries return zero rows:

TABLE	COMMAND	TARGET ROLE	POLICY CONDITION
`properties`	SELECT, UPDATE	Landlord	`landlord_id = auth.uid()`
`leases`	SELECT	Tenant	`tenant_id = auth.uid()`
`invoices`	SELECT	Tenant	`lease_id IN (SELECT id FROM leases WHERE tenant_id = auth.uid())`
`payments`	INSERT	Tenant	`tenant_id = auth.uid()`
`maintenance_tickets`	INSERT, SELECT	Tenant	`tenant_id = auth.uid()`
`utility_readings`	SELECT	Tenant	`unit_id IN (SELECT unit_id FROM leases WHERE tenant_id = auth.uid())`
`utility_readings`	ALL	Landlord	`property_id IN (SELECT id FROM properties WHERE landlord_id = auth.uid())`
`announcements`	SELECT	Tenant	`property_id IN (SELECT property_id FROM leases WHERE tenant_id = auth.uid())`

Automated RLS Regression Test Coverage

TEST SUITE	VERIFICATION SCOPE	ASSERTION EXPECTATION
`rls-tenant.test.ts`	Tenant isolation across units	Returns 403 on cross-tenant access.
`rls-landlord.test.ts`	Landlord property management	Disallows updates to foreign buildings.
`rls-bypass.test.ts`	Service role cron tasks	Successfully inserts recurring billing rows.
`rls-anon.test.ts`	Public and anonymous queries	Returns 0 rows for unauthenticated requests.
`rls-audit.test.ts`	Multi-tenant data isolation	Confirms zero cross-property data leakage.

> **Security Invariant: Row-level security is enforced at the PostgreSQL kernel level.**

Two-Factor Authentication (2FA) Lifecycle

iReside implements mandatory email-delivered 6-digit OTP verification for secure sessions:

1. User submits credentials at `/login` via Supabase Auth PKCE flow.
2. Server generates an ephemeral 6-digit cryptographic token (10-minute expiry).
3. Token is dispatched via transactional SMTP to the verified email.
4. User inputs token; upon verification, Supabase issues signed JWT and refresh tokens.

Authentication Threat Model & Defense Mechanisms

THREAT VECTOR	RISK LEVEL	DEFENSE ARCHITECTURE
Brute Force Login	High	Rate limit: 5 failed attempts per IP.
Session Hijacking	Critical	HTTP-only SameSite cookies and TLS 1.3.
OTP Replay Attack	Medium	10-minute expiry on single-use OTP codes.
Client Lock Stalls	High	Background non-blocking auth refresh execution.

Session Token Specifications & Lifecycle

TOKEN IDENTIFIER	STORAGE MEDIUM	VALIDITY WINDOW
Access Token (JWT)	Secure HTTP-only cookie	60 minutes with auto-refresh.
Refresh Token	Encrypted HTTP-only cookie	30 days rolling validity window.
Setup Cookie	Browser session cookie	Instant client navigation signal.
CSRF Token	Client Header	Protects state-changing REST endpoints.

Next.js Edge Middleware Evaluation Pipeline (middleware.ts)

ROUTE PATTERN	SESSION REQUIREMENT	MIDDLEWARE ACTION
`/landlord/*`	Valid session with Landlord role	Redirects to login if unauthenticated.
`/tenant/*`	Valid session with Tenant role	Redirects to login if unauthenticated.
`/setup/*`	Pending setup profile flag	Enforces onboarding wizard completion.
`/api/cron/*`	Valid CRON_SECRET header	Rejects missing tokens with HTTP 401.
`/login, /signup`	Anonymous / Unauthenticated	Redirects active sessions to home dashboard.

> **Client-side SDK token refreshes must NEVER be awaited on the critical navigation path.**

The Emergency Spare Key Architecture

The Emergency Spare Key provides failsafe account recovery when losing phone or email access:

Cryptographic Engine Specifications

PARAMETER	CRYPTOGRAPHIC STANDARD	OPERATIONAL IMPLEMENTATION
Entropy Source	CSPRNG 128-bit random bytes	Secure browser crypto API source.
String Format	16 uppercase alphanumeric chars	4 blocks separated by hyphens.
Hashing Algorithm	Bcrypt (Cost factor 12)	Plaintext key is never stored in DB.
Revocation Rule	Single-use consumption rule	Key is deleted upon successful recovery.

Account Recovery Verification Protocol

RECOVERY STEP	VERIFICATION CHECK	EXPECTED SYSTEM RESPONSE
1. Submit Email	Verifies user profile exists	Checks active user in PostgreSQL.
2. Submit Spare Key	Constant-time hash verification	Prevents timing side-channel attacks.
3. Session Reset	Revokes existing refresh tokens	Issues authenticated recovery session.
4. Key Rotation	Generates new spare key	Old key is rendered permanently invalid.

Atomic Recovery Lifecycle (/api/auth/spare-key-recovery)

The recovery endpoint validates the submitted key hash using constant-time verification.

Upon verification, existing tokens are revoked and a replacement key is generated.

Emergency Spare Key Error Handling & Failure Matrix

FAILURE SCENARIO	DETECTION MECHANISM	RECOVERY RESOLUTION RUNBOOK
Invalid Key Input	Constant-time hash comparison fails	Prompts retry with clear inline error message.
Consumed Key Reuse	Database flag checks key deletion	Rejects used key; guides user to support.
Rate Limit Exceeded	Exceeds 5 attempts per IP	Throttles requests for 15 minutes.

> Senior Landlord Invariant: Recovery keys use the Emergency Spare Key physical metaphor. Touch targets measure at least 48px.

The Turnkey Workspace Launch Lifecycle

The 5-stage onboarding pipeline guides new landlords through required system initialization:

5-Stage Onboarding Route Locking Matrix

STAGE	IDENTIFIER	ACCESSIBLE ROUTES	LOCKED ROUTES
Stage 1: Property	`no_property`	`/landlord/properties`, `/new`	All other sidebar navigation items.
Stage 2: Unit Map	`no_unit_map`	`/landlord/unit-map`	Operations and Finance workspaces.
Stage 3: Billing Rails	`no_billing_rails`	Dashboard, Properties, Utility	Finance Hub and Tenant records.
Stage 4: Tenants	`no_tenant`	Dashboard, Properties, Tenants	Deep operational financial pages.
Stage 5: Dashboard	`setupCompleted`	All production workspaces	None (All features fully unlocked).

Launch Atomic Validation Contract (/api/setup/launch)

VALIDATION CHECK	REQUIRED ENTITY	FAILURE BEHAVIOR
Property Exists	At least 1 registered property	Rejects launch with HTTP 400.
Units Configured	At least 1 configured rental unit	Rejects launch with HTTP 400.
Utility Rates Set	Non-zero water and electric rates	Rejects launch with HTTP 400.

Atomic Finalization Rail

When the landlord completes configuration, the workspace executes an atomic launch sequence:

1. Client submits setup state to `/api/setup/launch`.
2. Service role verifies property and utility rates exist.
3. Sets `is_setup_completed = true` in the profile database record.
4. Sets HTTP-only cookie `ireside_setup_completed=true` for immediate middleware evaluation.
5. Redirects to `/landlord/dashboard` via `router.replace`.

> **Navigation Rule:** Use router.replace to avoid back-navigation loops into setup wizards.

Core Application API Directory

iReside exposes structured REST endpoints for application and worker operations:

ROUTE	METHOD	AUTH	PAYLOAD & BEHAVIOR
`/api/setup/launch`	POST	Landlord	Stamps workspace completion and issues cookie signal.
`/api/cron/daily-billing`	POST	CRON_SECRET	Generates recurring invoices and assesses late fees.
`/api/auth/spare-key-recovery`	POST	Public	Redeems Emergency Spare Key for recovery session.
`/api/leases/e-sign`	POST	Tenant	Appends digital signature and countersigns lease.
`/api/payments/verify`	POST	Landlord	Verifies GCash reference number and settles invoice.
`/api/mobile/version`	GET	Public	Returns latest release APK version and download URL.
`/api/units/update-layout`	POST	Landlord	Saves 2D coordinates of room cards on unit map.
`/api/maintenance/update-status`	POST	Landlord	Advances repair work orders to completed status.

HTTP Status Code & Error Envelope Specifications

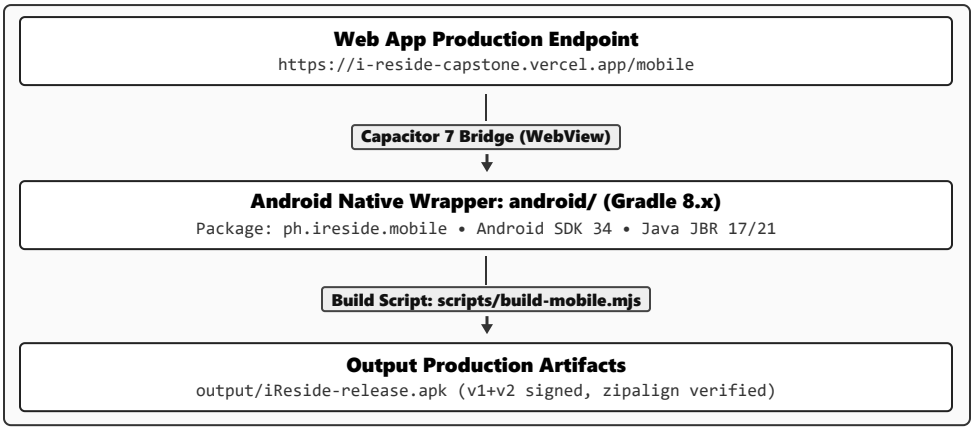
HTTP CODE	ERROR CODE	MEANING & CORRECTIVE DIRECTIVE
200 OK	`SUCCESS`	Request succeeded; payload delivered in data object.
400 Bad Request	`VALIDATION_ERROR`	Request payload failed schema validation rules.
401 Unauthorized	`SESSION_EXPIRED`	User session expired; re-authenticate at login.
403 Forbidden	`RLS_RESTRICTION`	Target record belongs to another user account.

API Rate Limiting & Throttling Thresholds

ENDPOINT GROUP	WINDOW SIZE	MAXIMUM REQUESTS ALLOWED
Authentication APIs	60 seconds	5 requests per IP address.
Public Onboarding Links	60 seconds	20 requests per IP address.
Standard App Workspaces	60 seconds	120 requests per IP address.

Mobile Client Architecture & Isolation

iReside provides a native Android experience via a standalone Capacitor 7 wrapper. Per `mobile-apk-build.md`, all native source is strictly isolated inside `android/`:



Android Build Toolchain Invariants & Paths

TOOLCHAIN COMPONENT	TARGET ENVIRONMENT	INVARIANT DIRECTIVE
Bundled JDK	Android Studio JBR 17/21	Incompatible system JDKs must not be used.
Android SDK	Android SDK API 34	Resolved from local user app data directory.
Gradle Wrapper	Gradle 8.x in android/	Native build logic remains inside android/.
Target URL	Production Vercel endpoint	Overridable via --url command flag.

Mobile Build Commands & Artifact Output

BUILD COMMAND	TARGET FUNCTION	OUTPUT FILE
`npm run mobile:build`	Compiles signed release APK	`output/iReside-release.apk`
`npm run mobile:clean`	Purges Gradle compilation cache	Cleans temporary build directories.

> **Zero Regression Invariant:** Mobile changes must pass Next.js build and TypeScript audits.

Vercel Serverless Cron Architecture

Automated processes run via Vercel Cron jobs declared in `vercel.json`:

JOB NAME	SCHEDULE	TARGET ROUTE	PURPOSE
`daily-billing`	`0 0 * * *` (Midnight UTC)	`/api/cron/daily-billing`	Generate recurring invoices and late payment penalties.
`lease-expiry-check`	`0 2 * * *` (02:00 UTC)	`/api/cron/lease-status`	Transition expired leases to ended status.
`payment-reminders`	`0 8 * * *` (08:00 UTC)	`/api/cron/reminders`	Dispatches rent reminders before due dates.
`purge-temp-tokens`	`0 4 * * 0` (Weekly Sun)	`/api/cron/purge-temp`	Cleans up expired onboarding invitations.

Billing Idempotency & Fault Tolerance Matrix

FAILURE MODE	DETECTION MECHANISM	RECOVERY RUNBOOK
Network Timeout	Vercel 15s execution ceiling	Worker resumes on next scheduled run.
Duplicate Trigger	Unique constraint on billing period	Database rejects duplicate insert attempts.
Database Unreachable	Supabase 503 HTTP status	Telemetry alert dispatched to operations.
Partial Batch Error	SQL transactional execution	Unprocessed leases are re-queued.

Cron Worker Security Validation Standards

SECURITY CHECK	EXPECTED VALUE	SECURITY POLICY
Authorization Header	`Bearer $CRON_SECRET`	Constant-time bearer token verification.
HTTP Method	HTTP POST	Non-POST requests rejected with HTTP 405.
User Agent Header	`vercel-cron/*`	Validated against serverless provider.

Cron Telemetry & Alerting Thresholds

AUTOMATION METRIC	TARGET THRESHOLD	AUTOMATED ALERTING POLICY
Cron Execution Duration	Under 10 seconds execution time	Alerts operations on execution timeouts.
Job Failure Rate	Zero consecutive job failures	Dispatches emergency SMS to on-call engineer.

Database Backup Tiers & Retention Policies

BACKUP TIER	FREQUENCY	RETENTION WINDOW	STORAGE LOCATION
WAL Archiving (PITR)	Continuous streaming	7 days rolling	Supabase multi-region storage.
Daily Snapshot Dump	03:00 UTC daily	30 days retention	Encrypted offsite cloud bucket.
Schema Migration History	Every git commit	Permanent	Versioned in git repository.
Weekly Offsite Backup	Sunday 04:00 UTC	90 days retention	Encrypted secondary cloud bucket.

Disaster Recovery Target Objectives (RTO & RPO)

DISASTER SCENARIO	TARGET RTO	TARGET RPO	RECOVERY PROCEDURE
Database Node Outage	≤ 15 minutes	≤ 1 minute	Automatic failover to replica node.
Accidental Row Deletion	≤ 30 minutes	≤ 5 minutes	Point-in-time recovery via Supabase CLI.
Failed Schema Migration	≤ 10 minutes	Zero data loss	Downward migration rollback script.

Infrastructure Telemetry & Alerting Thresholds

OPERATIONAL METRIC	WARNING THRESHOLD	ESCALATION ACTION
Database CPU Usage	Exceeds 80% for 5 minutes	Dispatches pager alert to engineering.
Connection Pool Level	Exceeds 90% capacity	PgBouncer auto-scales worker pool.
Server Error Rate	Exceeds 1% of total traffic	Vercel edge alerts operations team.

Disaster Recovery Step-by-Step Incident Runbook

RUNBOOK PHASE	PRIMARY OBJECTIVE	OPERATIONAL COMMAND
Phase 1: Triage	Identify outage root cause	Inspect Vercel and Supabase dashboards.
Phase 2: Isolation	Prevent dirty data writes	Enable maintenance mode in edge middleware.
Phase 3: Restore	Recover database to valid state	Execute point-in-time recovery via Supabase CLI.
Phase 4: Verify	Validate data integrity	Run automated verification test suite.

> **Maintenance Protocol:** Set maintenance flag in edge middleware during major restoration events.

Engineering Commissioning Sign-Off

This technical manual certifies that the iReside platform satisfies all architectural standards:

ROLE	NAME / ORGANIZATION	STATUS	SIGN-OFF DATE
Lead Systems Architect	iReside Engineering Group	Verified & Certified	Q1 2026
DevOps & Security Officer	Systems Security Committee	Production Approved	Q1 2026
Lead Mobile Engineer	Native Mobile Division	Verified & Signed	Q1 2026
Quality Assurance Lead	Validation & Test Operations	100% Suite Pass	Q1 2026

Automated Verification Suite Pass Record

VERIFICATION SUITE	EXECUTION COMMAND	RESULT STATUS
TypeScript Type Check	`npx tsc --noEmit`	Clean Pass (0 errors).
Validation Test Suite	`npx vitest run src/__tests__/validation`	Clean Pass (100% passing).
Production Next.js Build	`npm run build`	Clean Pass (Zero build errors).

Production Deployment Pipeline & Release Verification Protocol

DEPLOYMENT STAGE	AUTOMATED GATE	PASS CRITERIA
1. Continuous Integration	Automated GitHub Actions CI	Clean typecheck and 100% test pass.
2. Production Vercel	Edge Serverless Promotion	Zero route compilation errors.

Technical Glossary

RLS: Row-Level Security in PostgreSQL restricting row access per user session.
JWT: JSON Web Token used for cryptographically signed stateless session verification.
SERVICE ROLE: Privileged PostgreSQL role that bypasses RLS for administrative workers.
CAPACITOR 7: Cross-platform runtime executing the web app inside an Android WebView.
VERCEL CRON: Serverless scheduler invoking authenticated API endpoints at defined intervals.
BCRYPT: Adaptive cryptographic hash function used for Emergency Spare Key verification.
CSPRNG: Cryptographically Secure Pseudo-Random Number Generator.
PGBOUNCER: Lightweight connection pooler for PostgreSQL database.
WAL: Write-Ahead Logging ensuring durability and point-in-time recovery.
PKCE: Proof Key for Code Exchange securing authorization flows.
RTO: Recovery Time Objective specifying maximum allowable downtime.
RPO: Recovery Point Objective specifying maximum allowable data loss.

© 2026 iReside Technologies. All rights reserved. Confidential technical standard.